

SEUPD@CLEF: Team retrix on Source Retrieval for Scientific Web Claims

Fabio Baldan¹, Davide Donati¹, Alvisè Garberino¹, Giancarlo Padoan¹ and Marco Tessari¹

¹University of Padua, Italy

Abstract

We present the results of the retrix group for the CheckThat! 2026 - Task 1, in which the goal is to identify the scientific paper implicitly referenced in a social media post. The input consists of short, noisy and usually informal posts that mention scientific findings without providing explicit links or explicit citations. In our pipeline, first we translate the non-English posts, then we expand them using an embedding model and the terms of the top 5 documents (ranked by similarity). After that, the first research is an hybrid retrieval system that combines dense FAISS search with sparse lexical matching and ColBERT multi-vector scoring ($s = 0.5 s_{\text{dense}} + 0.15 s_{\text{sparse}} + 0.35 s_{\text{colbert}}$). Finally a neural re-ranking with a fine-tuned cross-encoder model is performed. On the development set our best system achieves an average MRR@5 of 0.71 (second place in the official dashboard).

Keywords

Fact Checking, Information Retrieval, Scientific Document Retrieval, Social media fact verification, CLEF, Neural Re-ranking, Cross Encoder

1. Introduction

The rapid spread of scientific claims through social media creates a difficult fact-checking scenario: posts often report, simplify, or sensationalize research findings while omitting the original source. In many cases, the cited study is not linked explicitly with a URL or a bibliographic reference, but only mentioned indirectly through fragments such as the reported finding, the venue, the institution, or the authors. As a consequence, verifying such claims requires first identifying the scientific publication that is being implicitly referenced.

This problem is the focus of Task 1 of the CLEF 2026 CheckThat! Lab, *Source Retrieval for Scientific Web Claims*. Given a social media post containing a scientific claim and an implicit reference to a paper, the goal is to retrieve the correct publication from a fixed collection of 10.000 candidate papers written in English.

The task is challenging for several reasons. First, the linguistic gap between social media and scientific writing is substantial: posts are short, informal and often contain paraphrases, slogans, hashtags or incomplete statements, whereas scientific papers use more precise and technical language. Second, the reference to the source paper is implicit rather than explicit so the retrieval system must infer the connection from partial semantic evidence rather than from direct citations. Third, relevant clues may appear in different forms, such as mentions of a scientific result, a medical outcome, a research institution, or a publication venue, requiring the system to combine lexical and semantic evidence effectively.

An additional challenge is multilinguality: the official query sets include English, German and French posts while the candidate pool remains shared across languages.

Team Retrix approaches this problem with a four-stage sequential pipeline.

1. In a preliminary stage we translate to English all the queries written in French and German

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2025/2026

✉ fabio.baldan.1@studenti.unipd.it (F. Baldan); davide.donati@studenti.unipd.it (D. Donati);
alvisè.garberino@studenti.unipd.it (A. Garberino); giancarlo.padoan@studenti.unipd.it (G. Padoan);
marco.tessari.8@studenti.unipd.it (M. Tessari)



© 2026 Copyright for this paper by its authors.
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Our work is also related to the broader literature on automatic claim verification and, more specifically, to the CheckThat! shared tasks, which have addressed the detection, verification, and analysis of claims in social media across several editions [7, 8, 9]; however, source retrieval for scientific web claims differs from standard verification tasks in an important way: the system is not asked to classify a claim as true or false, but to recover the original scientific paper implicitly referenced in the post and for this reason, the task is closely connected to evidence retrieval, while also requiring robust matching between informal multilingual posts and formal scientific documents.

3. Dataset

The dataset used in this work is organized as a source-retrieval task in which, given a social-media post expressing a scientific claim and implicitly referring to a scientific publication without linking to it explicitly, the system is required to identify the corresponding paper within a fixed pool of candidate documents. The benchmark is multilingual on the query side, as posts are provided in English, German, and French, whereas all queries are matched against the same shared collection of scientific papers.

The document collection consists of 10,000 scientific publications, each identified by a unique pubkey and described through the fields `title`, `abstract`, `venue`, and `authors`; in our pipeline, each candidate document is represented by the concatenation of title and abstract, a choice that provides a compact yet informative textual representation for both retrieval and reranking, while preserving a uniform document space across all languages.

The query data are organized by language and by split: for English, German, and French, both the training and development portions contain the query identifier (`index`), the post text, and the gold pubkey of the referenced paper, thus supporting supervised training as well as model selection and internal evaluation, the released test files, instead, include only the query identifier and the post text, without the corresponding gold document identifier.

The Table 1 reports the size of all the data splits considered in our experiments.

Table 1: Dataset statistics for the version used in our experiments.

Split	EN	DE	FR	Total
Train	14,977	1,460	2,807	19,244
Dev	3,905	386	702	4,993
Test	6,076	876	1,220	8,172

This setting is particularly challenging because the queries take the form of short, noisy, and informal social-media posts, while the target documents are substantially longer scientific texts written in a more formal and information-dense style. As a result, the system must bridge a marked gap in both length and linguistic register between the two sides of the retrieval problem; moreover, the multilingual nature of the benchmark further increases the difficulty, since queries in different languages must all be matched against the same shared document collection, requiring the model to combine semantic matching with cross-lingual generalization.

4. Methodology

We started from a lexical baseline based on Apache Lucene BM25 and progressively moved to a multi-stage neural pipeline. Early experiments focused on analyzer choices, stopword lists, and BM25 variants; these runs were useful to establish a stable baseline, but they also showed that lexical overlap alone is insufficient for this task.

We adopted a four-stage pipeline (Figure 1) for our information retrieval system, consisting of:

- **Query Translation:** queries in French or German are first translated into English.

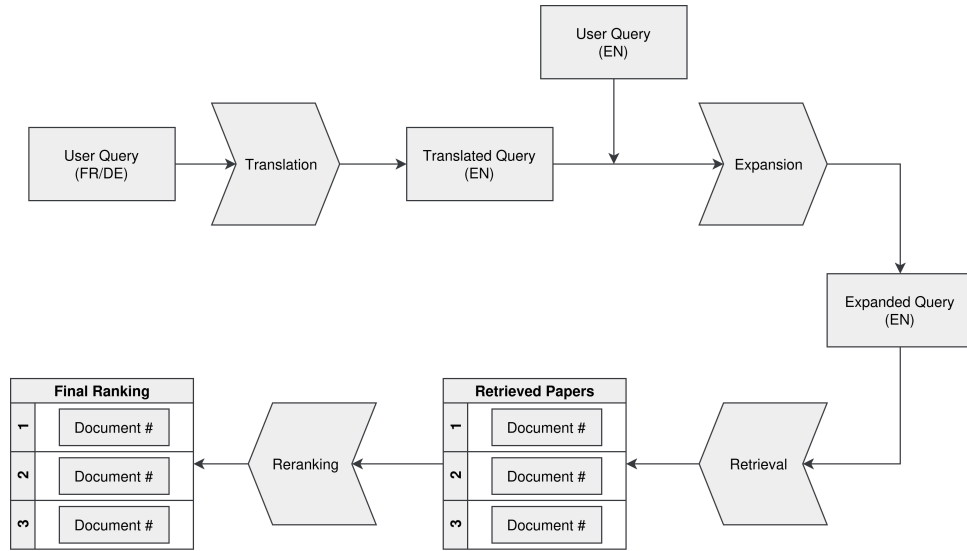


Figure 1: Four-stage pipeline for information retrieval.

- **Query expansion:** we expanded the English query with additional context to bridge the gap between informal and domain specific language.
- **Retrieval:** we retrieved relevant documents from the corpus using the expanded query.
- **Reranking:** we reranked the retrieved documents based on their relevance to the original query.

For each of these stages, we experimented with different models, choosing the ones that performed better in relation to the others.

In the following sections all the four stages are precisely explained.

4.1. Query translation

In the preliminary stage of our pipeline non-English queries (French and German) are translated into English. This is done query by query using the model `utter-project/EuroLLM-9B-Instruct-2512` with the following prompt:

LLM Prompt

```

system: You are a professional translator.
Translate the user's text to English.
Output ONLY the English translation.
Do not include the original text, explanations, labels, or any other content.

user: Translate this {French/German} text to English.
Output only the English translation, nothing else.

{Query original text}
  
```

The resulting English queries are then fed into the following step of our pipeline.

4.2. Query Expansion

Following the translation phase, the English queries undergo a semantic and lexical expansion process. This stage is designed to bridge the vocabulary gap between the user's informal query and the formal academic language present in the scientific document corpus. Unlike previous iterations that relied on generative LLM re-phrasings, the current architecture utilizes a hybrid expansion strategy, combining deep semantic retrieval with linguistic analysis. The expansion process follows three main technical steps:

- **Keyword Extraction:** We use the spaCy library (en_core_web_sm) to perform Part-of-Speech (POS) tagging on the translated query. We specifically extract nouns (NOUN) and proper nouns (PROPN), which represent the core technical entities of the claim.
- **Semantic Neighbor Retrieval:** The query is embedded using the BAAI/bge-large-en-v1.5 model. We retrieve the top-5 most similar documents from the corpus to act as a source of domain-specific terminology. To ensure high-quality embeddings, we apply the BGE-specific instruction prefix: *"Represent this sentence for searching relevant passages:"*.
- **Term Frequency Selection:** Candidate expansion terms are extracted from the retrieved neighbors and weighted based on their frequency and the original semantic similarity score.

The final expanded query is constructed by concatenating the original translated text, the extracted POS keywords, and the top-15 most relevant expansion terms. This creates a high-density representation that maximizes the probability of overlap during the hybrid retrieval stage, ensuring that both specific technical nomenclature and broader semantic contexts are represented. The resulting JSON object preserves the original query metadata while adding an expanded field for the downstream retrieval framework.

Before confirming this as query expansion strategy, we have also tried to use two LLM models, meta-llama/Llama-3.2-1B and openai/gpt-oss-20b with the following prompt:

LLM Prompt

```
system: "Reasoning: {reasoning_effort}"
You are a strict JSON-only retrieval query expansion generator.
Return the final answer only as valid JSON.

user: You generate retrieval-specific query expansions.
Return only one valid JSON object with exactly these string fields:
{
  "keywords": "...",
  "sparse": "...",
  "embedding": "...",
  "colbert": "...",
  "expanded": "..."
}

Rules:
- Do not include markdown, code fences, emoji, commentary, arrays, or nested objects.
- All five values must be strings.
- Do not hallucinate facts.
- Do not translate unless translation is already present in the query.
- Preserve original-language terms.
- keywords: compact important terms only; entities, nouns, technical terms, names, locations, dates, identifiers, acronyms, modifiers; no sentence.
- sparse: lexical BM25/SPLADE-style expansion; exact-match terms, synonyms, acronyms, aliases, spelling and morphology variants; phrase fragments separated by spaces or semicolons; no long prose.
- embedding: concise HyDE/query2doc-style natural-language semantic pseudo-document; include intent, context, and likely relevant-document framing.
- colbert: concise late-interaction expansion; preserve important tokens and entities; add focused context and aliases; avoid synonym spam.
- expanded: readable general-purpose merged expansion string combining the useful hints from the other fields.
```

This approach, which used a different expansion for each retrieval representation, did not improve performance. We therefore retained the simpler and more effective configuration based on a single expanded query. Section 4.3 describes in more detail how these expansion strategies were evaluated

during retrieval.

4.3. Retrieval

The retrieval stage is designed as a high-recall candidate-generation step between query expansion and neural reranking. Its objective is not to make the final relevance decision, but to ensure that the true source paper is included in a sufficiently large candidate set for the subsequent, more computationally expensive cross-encoder stage. For this reason, after experimenting with a Lucene BM25 baseline, BGE-M3 using dense vectors only, and other bi-encoder models, the final pipeline adopts **BAAI/bge-m3** in a hybrid retrieval setting.

This choice reflects the nature of the task. Social media claims often contain informal paraphrases, hashtags, named entities, numerical values, and fragments of scientific terminology. Therefore, relying exclusively on exact lexical overlap or exclusively on dense semantic similarity may discard useful evidence. Hybrid retrieval is better suited to this setting because it combines complementary retrieval signals and can offer higher accuracy and stronger generalization capabilities [10].

The retriever searches the fixed paper collection provided by the task organizers. Each paper is represented at document level by concatenating its title and abstract, and the retrieved unit is always the paper pubkey. We did not split papers into smaller chunks because the task requires identifying the source publication rather than retrieving a specific evidence passage. Using one representation per paper also avoids an additional chunk-to-document aggregation step. Although the collection includes metadata such as venue and authors, this hybrid retrieval stage uses only titles and abstracts, since these are the fields consistently encoded for all documents.

All input queries are converted into plain English, either because they are originally written in English or because they are translated before retrieval, see Section 4.1. They are then expanded using the procedure described in Sections 4.2.

We chose **BAAI/bge-m3** because it is well suited to this retrieval setting. The model can produce dense vectors, sparse lexical weights, and ColBERT-style multi-vector representations from the same encoder. This allows the retriever to combine broad semantic matching, exact term evidence, and fine-grained token-level matching without switching between unrelated encoders.

Candidate selection is performed in two steps. First, all documents are encoded as dense BGE-M3 vectors and searched with an inner-product FAISS index [1]. This dense stage acts as an efficient broad filter over the whole collection. The default setting retrieves a large candidate pool of $\text{candidate_multiplier} \times \text{top_k}$, with $\text{candidate_multiplier} = 3$ and $\text{top_k} = 2000$, before the final truncation. This deliberately large candidate set favours recall, which is more important than precision at this stage.

Second, the dense candidates are rescored using the sparse and ColBERT-style representations. The sparse component rewards important overlapping terms, while the ColBERT-style component captures local token correspondences that may be lost when each document is represented by a single dense vector [5].

The final configuration was selected through a series of development runs. We first tuned the lexical baseline by varying the Lucene analyzer, stop-word lists, stemming strategy, and BM25 searcher variants. These experiments improved the classical baseline, but also highlighted its limitations for this task: even with query expansion and reranking, BM25-based candidate generation remained more dependent on surface-level lexical overlap than desired.

We then evaluated dense retrieval with several bi-encoder models. Among these, **BAAI/bge-m3** achieved the strongest performance and improved the semantic coverage of the candidate set, particularly for paraphrased and multilingual claims. We also tested BGE-M3 in a hybrid retrieval setting, using a setup that differed slightly from the final configuration. In those experiments, sparse, dense, and ColBERT-style retrieval were performed using dedicated query expansions tailored to each retrieval mode. More specifically, each query was represented through three complementary views: an embedding view for semantic matching, a sparse view for lexical and term-based evidence, and a ColBERT view for fine-grained token-level interaction.

However, the best results in our analysis were obtained with a simpler configuration in which BGE-M3 hybrid retrieval used the same expanded query, described in the previous section, for all three retrieval modes. We then tested several weighting configurations. Among them, the run labelled `bge_m3_hybrid_513` was retained because it provided the strongest recall-oriented trade-off among the retrievers evaluated without reranking. This configuration used weights of 0.5, 0.15, and 0.35 for dense, sparse, and ColBERT scoring, respectively, with a multiplier of 3 and a maximum hybrid input length of 384.

A closely related configuration with a larger ColBERT weight achieved a nearly identical MRR@5, but a lower Recall@100. We therefore preferred the configuration that better matched the role of retrieval as a candidate generator. This choice was further confirmed by downstream experiments: reranking candidates from `bge_m3_hybrid_513` consistently outperformed the corresponding BM25-based reranked runs. In addition, increasing the reranked candidate depth from a few hundred documents to between 1000 and 2500 documents improved the final recall available to the reranker.

We also experimented with a learned fusion strategy for the hybrid signals. In particular, the experimental setup included a LightGBM LambdaMART option that builds features from the dense, sparse, and ColBERT scores, together with normalized score variants and dense-rank features, and then learns how to combine them. However, the retained retrieval configuration uses simpler linear fusion, which is easier to reproduce and aligns well with the recall-oriented role of this stage.

The final hybrid retrieval score is therefore computed as:

$$s(q, d) = 0.5 s_{\text{dense}}(q, d) + 0.15 s_{\text{sparse}}(q, d) + 0.35 s_{\text{colbert}}(q, d).$$

The largest weight is assigned to the dense score because it provides the main semantic signal. The sparse and ColBERT components add complementary lexical and fine-grained matching evidence. Candidates are sorted by this score in descending order and truncated to the final `top_k`. The retrieval output is a JSON mapping from each query ID to an ordered list of paper pubkeys. Scores are not exported, because the next stage treats retrieval as a candidate generator and recomputes relevance using a cross-encoder over the returned document identifiers.

4.4. Reranking

The reranking stage receives the candidate list produced by the hybrid retriever and reorders it with cross-encoders. Its goal is to improve early precision (especially MRR@5) while preserving as much recall as possible from the first stage.

Our strongest single-stage reranker is `nvidia/llama-nemotron-rerank-1b-v2`. For each query-document pair, we build an input sequence with the template `question: {q} followed by passage: {p}` and compute a scalar relevance score with a sequence-classification head. Documents are then sorted by this score. In practice, reranking deeper candidate pools (e.g., 500–1000 documents) gave better final results than shallow pools, confirming the importance of high recall before neural ranking.

We also evaluated alternative rerankers, including Qwen3 and GTE-ModernBERT variants, and tested cascaded reranking. In the best cascaded configuration, Nemotron is used as the first reranking pass and a second reranker (`BAAI/bge-reranker-v2-m3` or a fine-tuned `ModernBERT-large`) refines the top portion of the list. This two-step reranking improves top-rank ordering over single-stage reranking.

4.5. Evaluation

We evaluate systems on the CheckThat! Task 1 query sets by matching retrieved pubkeys against the gold pubkey for each query. The official optimization target is MRR@5, which we use as the primary score during model and parameter selection. In parallel, we track Recall@100 to monitor candidate-generation quality and to verify that reranking gains do not come from overly aggressive filtering.

For internal analysis, our evaluator also computes additional ranking metrics (e.g., Recall@1/5/10/100, Precision@1/5/10, MAP, and nDCG@k). This broader view is useful to interpret trade-offs between

recall-oriented retrieval settings and precision-oriented reranking settings, while keeping MRR@5 as the main criterion for reporting and model selection.

5. Experimental Setup

We use the official CheckThat! 2026 Task 1 collection format: social-media claims as queries and a fixed paper corpus in `code/data/collection_data.json`. Query files are handled in JSON format (English, German, and French) and expanded versions are generated in `code/data/expanded_queries_multilingual_`. Retrieved outputs are always ranked lists of paper pubkeys.

Our primary evaluation measure is MRR@5, consistent with the task objective and the official scorer used in `results/scoring_official/`. During development, we also monitor Recall@100 and additional internal metrics from the Java evaluator (Recall@k, Precision@k, MAP, nDCG@k) to analyze recall/precision trade-offs across retrieval and reranking configurations.

The implementation is organized into: (i) Java components for indexing, baseline retrieval, and evaluation (`code/src/main/java/unipd/se/`); (ii) Python components for multilingual query expansion, hybrid BGE-M3 retrieval, and neural reranking (`code/src/main/java/unipd/se/py/`); and (iii) run outputs and metrics under `results/`. The project repository is available at <https://bitbucket.org/upd-dei-stud-prj/seupd2526-retrix.git>.

In terms of execution environment, lightweight preprocessing and analysis can run on CPU, while the neural stages (LLM-based expansion, hybrid BGE-M3 encoding, and cross-encoder reranking) are configured to use CUDA GPUs when available, with automatic CPU fallback paths implemented in the code.

6. Results and Discussion

6.1. Main Results

Table 2 summarizes ten representative runs selected from our experimental campaign; rather than listing the top-scoring runs only, we chose configurations that cover the main families of experiments explored during the project, namely analyzer tuning, retrieval-only hybrid search, single-stage reranking, alternative rerankers, and cascaded reranking.

The results in Table 2 show a clear progression across the different stages of the project. The best retrieval-only configuration reaches an MRR@5 of 0.557, while the best cascaded system reaches 0.766. The largest individual improvement comes from adding `nvidia/llama-nemotron-rerank-1b-v2` as the first reranking stage, which moves the system from the 0.557 range of the hybrid-only runs to 0.702. A second reranking stage then brings a further gain, with the fine-tuned ModernBERT model producing the strongest final configuration.

6.2. Ablation / Run Analysis

Analyzer tuning: We initially explored stronger English analyzers and several stemming and stop-word variants in order to improve lexical matching between short social-media posts and scientific abstracts. This family peaks at an MRR@5 of 0.503 with `MyEnglishAnalyzer_v2`, while the other analyzer variants remain around the 0.45–0.50 range. These runs were useful in validating the pre-processing choices, but they also showed that text normalization alone is not enough to bridge the semantic gap between informal claims and academic documents. Analyzer tuning yields measurable improvements, but its impact is limited when compared with semantic retrieval and neural reranking.

Hybrid retrieval without reranking: A shift from analyzer-focused experiments to hybrid retrieval leads to a substantially larger improvement; the two best retrieval-only systems, reported in Table 2 as Hybrid A and Hybrid B, both reach an MRR@5 of about 0.557, with recall@100 between 0.837 and 0.839. The difference between them lies in the balance between the dense and ColBERT components, while

Table 2
Representative runs selected from the experimental campaign.

Family	Run label	Key configuration	MRR@5	Recall@100
Analyzer tuning	Analyzer V2	Lexical retrieval with MyEnglishAnalyzer_v2 and BGE-V1 expansion	0.503	0.796
Analyzer tuning	Analyzer V1.12	Lexical retrieval with KStem and custom stoplist variant	0.450	0.783
Retrieval only	Hybrid A	BGE-M3 hybrid retrieval, dense 0.35, sparse 0.15, ColBERT 0.50	0.557	0.837
Retrieval only	Hybrid B	BGE-M3 hybrid retrieval, dense 0.50, sparse 0.15, ColBERT 0.35	0.557	0.839
Single-stage reranking	Nemotron top-400	Hybrid retrieval followed by Nemotron reranking on 400 candidates	0.694	0.893
Single-stage reranking	Nemotron top-1000	Hybrid retrieval followed by Nemotron reranking on 1000 candidates	0.702	0.911
Alternative rerankers	Qwen3-4B	Hybrid retrieval followed by Qwen3 reranking	0.662	0.839
Alternative rerankers	GTE-ModernBERT	Hybrid retrieval followed by GTE-ModernBERT reranking	0.652	0.852
Cascaded reranking	Nemotron + BGE-v2-M3	Nemotron first-stage reranking followed by BGE reranking	0.741	0.893
Cascaded reranking	Nemotron + ModernBERT-FT	Nemotron first-stage reranking followed by fine-tuned ModernBERT reranking	0.766	0.893

the sparse contribution is kept fixed. This indicates that combining dense, sparse, and ColBERT-style signals already produces a strong candidate set before any reranking is applied; at the same time the very small difference between the two configurations suggests that, once the hybrid formulation is in place, the exact internal weighting matters less than the transition away from purely lexical retrieval. Hybrid retrieval is the first major step change in effectiveness and provides the strongest retrieval-only foundation for the full pipeline.

Single-stage reranking with Nemotron: Adding `nvidia/llama-nemotron-rerank-1b-v2` on top of the hybrid candidates produces the largest isolated gain in MRR@5; the representative Nemotron runs increase from the 0.557 range of the retrieval-only systems to 0.694 and 0.702, while recall@100 also rises from about 0.839 to as high as 0.911. We also observe a limited but consistent benefit from increasing the number of candidates exposed to the reranker: moving from the top-400 to the top-1000 setting slightly improves both MRR@5 and recall@100; this suggests that the reranker can exploit a broader candidate pool without being significantly affected by additional noise. The first reranking stage is the most impactful component of the pipeline, and a larger candidate pool brings further, although diminishing, gains.

Alternative rerankers: To verify that the gains were not tied to a single model, we also evaluated other rerankers, including `Qwen3-reranker-4B` and `Alibaba-NLP/gte-reranker-modernbert-base`; both models clearly outperform the retrieval-only baseline in terms of MRR@5, reaching values of 0.662 and 0.652, respectively. However, they remain below the Nemotron runs, especially on MRR@5, which suggests weaker ranking quality in the first positions of the final list; even so these experiments confirm that the hybrid retriever produces candidate sets that can be improved by different neural rerankers. Alternative rerankers validate the overall pipeline design, but Nemotron is the strongest first-stage reranker among the models we tested.

Cascaded reranking: Our best results are obtained by applying a second reranking stage to the candidates already filtered by Nemotron; the cascade based on `BAAI/bge-reranker-v2-m3` reaches an MRR@5 of 0.741, while the best overall system, based on a fine-tuned version of `answerdotai/`

ModernBERT-large, reaches 0.766. Both cascaded runs keep recall@100 at 0.893, which is slightly lower than the Nemotron top-1000 configuration but still substantially higher than the retrieval-only systems; this indicates that the second reranker mainly improves the ordering of already strong candidates rather than increasing the breadth of the candidate pool. Cascaded reranking is the best strategy to maximize early precision, and the fine-tuned ModernBERT reranker provides the strongest final ranking.

7. Conclusions and Future Work

This project explored a practical, reproducible pipeline for document retrieval built around three complementary stages:

1. Query expansion: LLM-based and PRF-style;
2. High-recall candidate generation via a hybrid BGE-M3 retriever that fuses dense, sparse and ColBERT-like signals;
3. Neural reranking where a cross-encoder (a fine-tuned version of `nvidia/llama-nemotron-rerank-1b-v2`) is applied to refine the top candidates.

Our empirical findings and practical experience can be summarized as follows:

- Neural reranking yields the largest single improvement in ranked quality: applying a cross-encoder to a high-recall candidate set substantially increases MRR@5 and precision at small cutoffs compared to lexical baselines alone. This improvement is even more noticeable if the re-ranking model is fine-tuned.
- A hybrid retrieval approach (dense + sparse + ColBERT) produces a robust candidate set that improves downstream re-ranker performance, reducing significantly the inference time to produce the re-ranked results while maintaining good overall performance. Careful weighting of the signals and the use of a candidate multiplier were important to maintain recall.
 ??????????????FINIRE CON RISULTATIIII SU RUN ??????????????FINIRE CON RISULTATI-
 IIII ??????????????FINIRE CON RISULTATIIII ??????????????FINIRE CON RISULTATIIII
 ??????????????FINIRE CON RISULTATIIII
- Hybrid retrieval approach is a valuable resource and even re-ranking on a large number of the first top-k elements doesn't enhance MRR@5 by a significant value while inference time grows significantly. Here comes into play the quality of the retrieval model to discern noise from important documents. For example for Alibaba-NLP/gte-reranker-modernbert-base [11] (0.1B parameters) this value is around 130 (+0.012 and +0.002 MRR@5 compared to the run with top-k=100 from experimental tries in our system) while for `nvidia/llama-nemotron-rerank-1b-v2`[12] this value is around 2000 (+0.015 Recall@100 and +0.004 MRR@5 if compared to the run with top-k=100 but passed from 5:30:00 hours to generate the re-ranked results for the topk=100 to more than 48 hours for topk). If you tried to go over this values the MRR@5 began to decrease.
- Traditional analyzer and BM25 tuning remain valuable: they provide a competitive, computationally cheap baseline and act as a useful fallback when GPU resources are limited.

7.1. Limitations and future work

The experiments reported in this repository are exploratory and optimized for research reproducibility rather than for production latency. The lack of suitable hardware components has hindered the project's development, imposing time constraints and limitations on the size of the models used. Looking ahead there are several areas for future development. First of all a first fine-tuning of the embedding model used and a better one for the re-ranker model would certainly increase the performance of the system. The opportunity to have the datasets of the last year CLEF [13] that were very similar to the ones of this year avoided us to use techniques as data augmentation but it would be interesting to discover if they could have improve our system's performance. A more in-depth studying of hard-negatives techniques for the fine-tuning of the re-ranker would also be interesting such as "Hard Negative Mining for Domain-Specific Retrieval in Enterprise Systems"[14] and others.

8. Declaration on Generative AI

During the writing of this paper the authors used Grammarly and DeepL for grammar and spelling checks and Claude Sonnet 4.6 for writing-style improvement, citation management and formatting assistance. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

9. Group Members Contribution

Fabio Baldan has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Writing and revising some chapters of the documentation based on the actual code and on the concepts learned during the project development;
3. Tracking and reviewing the annotation-part for the runs with various systems;
4. Trying various models for the re-rank part to enhance the MRR@5;
5. Implementing a custom evaluator to focus on metrics that matter the most;
6. Reviewing and improving the query-expansion;
7. Fine-tuning the re-ranker to improve the MRR@5 and assure an advantage if compared with the default one.

Davide Donati has contributed to this project by:

1. Helping designing and implementing the initial BM25 basic pipeline;
2. Implementing a BooleanQuery-based parsing in `Searcher.java` to separate the original tweet and expanded terms into different clauses with tailored boosts (and applying field filters for venue/authors), improving retrieval precision;
3. Implementing and testing the multilingual query-expansion pipeline (e.g. `query_expansion_multilingual` tested with `PROVEDAVIDE.py`) and fixing issues found during integration;
4. Adding a PRF (pseudo-relevance feedback) implementation and accompanying AI-assisted documentation/examples (PRF code is available in the discarded-code area for reference and analysis);
5. Contributing to documentation and write-ups;
6. Keeping track of experimental runs and results for the systems he developed and tested.

Alvise Garberino has contributed to this project by:

1. Helping design and implement the initial BM25-based retrieval pipeline;
2. Helping and searching the best models and techniques to implement a re-ranker model;
3. Implementing and improving the re-ranker fine-tuning pipeline, including data augmentation through back-translation, hard-negative construction/mining, and data-distillation support;
4. Contributing to the evaluation workflow by tracking experimental runs, analyzing results, and supporting checkpoint evaluation;
5. Contributing to the Java support code and documentation, including refactoring/query-model improvements and JavaDoc updates;
6. Writing and revising parts of the documentation, especially the introduction, related work, results, conclusion, and experimental setup sections, while also selecting and integrating the corresponding academic references.

Giancarlo Padoan has contributed to this project by:

1. Designing, implementing and fixing the initial basic pipeline;

2. Writing and correcting some chapters of the documentation based on the actual code and on the notions learned while building the project;
3. Designing and implementing an hybrid-retrieval system to enhance recall, which is the core of the final architecture;
4. Trying various models for the embedding-representation part to enhance the MRR@5;
5. Implementing the first version of query-expansion functionality, and improving it by testing different techniques;
6. Tested different scoring function for combining the hybrid-retrieval scores;
7. Testing various cross-encoder models for the re-ranking stage of the pipeline;
8. Keeping track of the results of the systems that he tried.

Marco Tessari has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Improved the performance of the English Analyzer (text-processing, stemmer, filters and stopwords);
3. Writing part of the docs and helping other members by reviewing their writes;
4. Keeping track of the results of the systems that he tried;
5. Helping and searching the best models and techniques to implement an embedding model;
6. Experimenting with word2vec static embeddings for query expansion;
7. Helping to find the best weights to enhance recall in the hybrid-retrieval system.

References

- [1] Faiss, Faiss documentation, <https://faiss.ai/>, ???? Accessed: 01-05-2026 (DD-MM-YYYY, italian date format).
- [2] S. E. Robertson, U. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *Foundations and Trends in Information Retrieval (FnTIR)* 3 (2009) 333–389.
- [3] S. Marchesin, A. Purpura, G. Silvello, Focal elements of neural information retrieval models. An outlook through a reproducibility study, *Information Processing & Management* 57 (2020) 102109:1–102109:29.
- [4] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense Passage Retrieval for Open-Domain Question Answering, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: <https://aclanthology.org/2020.emnlp-main.550/>. doi:10.18653/v1/2020.emnlp-main.550.
- [5] O. Khattab, M. Zaharia, ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT, in: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2020)*, ACM Press, New York, USA, 2020, pp. 39–48. URL: <https://doi.org/10.1145/3397271.3401075>. doi:10.1145/3397271.3401075.
- [6] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document Ranking with a Pretrained Sequence-to-Sequence Model, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*, Association for Computational Linguistics, Online, 2020, pp. 708–718. URL: <https://aclanthology.org/2020.findings-emnlp.63/>. doi:10.18653/v1/2020.findings-emnlp.63.
- [7] T. Elsayed, P. Nakov, A. Barrón-Cedeño, M. Hasanain, R. Suwaileh, G. Da San Martino, P. Atanasova, Overview of the CLEF-2019 CheckThat!: Automatic Identification and Verification of Claims, in: F. Crestani, M. Braschler, J. Savoy, A. Rauber, H. Müller, D. E. Losada, G. Heinatz Bürki, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Tenth International Conference of the CLEF Association (CLEF 2019)*, Lecture Notes in Computer Science (LNCS) 11696, Springer, Heidelberg, Germany, 2019, pp. 301–321.

- [8] A. Barrón-Cedeño, T. Elsayed, P. Nakov, G. Da San Martino, M. Hasanain, R. Suwaileh, F. Haouari, N. Babulkov, B. Hamdan, A. Nikolov, S. Shaar, Z. Sheikh Ali, Overview of CheckThat! 2020: Automatic Identification and Verification of Claims in Social Media, in: A. Arampatzis, E. Kanoulas, T. Tsikrika, S. Vrochidis, H. Joho, C. Lioma, K. Eickhoff, A. Névél, L. Cappellato, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Eleventh International Conference of the CLEF Association (CLEF 2020)*, Lecture Notes in Computer Science (LNCS) 12260, Springer, Heidelberg, Germany, 2020, pp. 215–236.
- [9] P. Nakov, G. Da San Martino, T. Elsayed, A. Barrón-Cedeño, R. Míguez, S. Shaar, F. Alam, F. Haouari, M. Hasanain, W. Mansour, B. Hamdan, Z. S. Ali, N. Babulkov, A. Nikolov, G. K. Shahi, J. M. Struß, T. Mandl, M. Kutlu, Y. S. Kartal, Overview of the CLEF–2021 CheckThat! Lab on Detecting Check-Worthy Claims, Previously Fact-Checked Claims, and Fake News, in: K. S. Candan, B. Ionescu, L. Goeuriot, B. Larsen, H. Müller, A. Joly, M. Maistro, F. Piroi, G. Faggioli, N. Ferro (Eds.), *Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Twelfth International Conference of the CLEF Association (CLEF 2021)*, Lecture Notes in Computer Science (LNCS) 12880, Springer, Heidelberg, Germany, 2021, pp. 264–291.
- [10] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. [arXiv:2402.03216](https://arxiv.org/abs/2402.03216).
- [11] Tongyi Lab, Alibaba Group, gte-reranker-modernbert-base: A text reranking model built upon ModernBERT for efficient information retrieval, <https://huggingface.co/Alibaba-NLP/gte-reranker-modernbert-base>, 2023. Accessed: [Inserire la data di consultazione, es. 2026-05-06].
- [12] NVIDIA, Llama Nemotron Reranking 1B v2: A multilingual text reranking model for question-answering retrieval, <https://huggingface.co/nvidia/llama-nemotron-rerank-1b-v2>, 2026. Accessed: [Inserire la data di consultazione, es. 2026-05-06].
- [13] CheckThat! Lab, CLEF2025 CheckThat! Lab: Multimodal, Multilingual, and Explainable Fact-Checking, https://gitlab.com/checkthat_lab/clef2025-checkthat-lab, 2024. Accessed: [Inserire la data di consultazione, es. 2026-05-06]. Il repository contiene i task, i dati e le risorse per la competizione CLEF 2025 sul fact-checking.
- [14] H. Meghwani, A. Agarwal, P. Pattnayak, H. L. Patel, S. Panda, Hard negative mining for domain-specific retrieval in enterprise systems, 2025. URL: <https://arxiv.org/abs/2505.18366>. [arXiv:2505.18366](https://arxiv.org/abs/2505.18366).